

Experiment 1

Classes, Constructors and Pointers in C++

OBJECTIVE:

- Define and use classes in C++
- Implement constructors
- Create objects on stack and heap
- Understand pointer fundamentals
- Use dynamic memory allocation (new and delete)
- Use pointer to object and arrow operator (->)

INTRODUCTION:

Before studying advanced data structures such as Linked Lists, Trees, and Graphs, it is essential to understand how classes, objects, pointers, and dynamic memory work in C++. This lab provides the foundation required for implementing dynamic data structures.

INTRODUCTION TO CLASSES

A class in C++ is a user-defined data type that acts as a blueprint for creating objects. It allows us to combine data and functions into a single unit.

- A class defines what an object contains (data members).
- A class defines what an object can do (member functions).
- An object is an instance of a class.

BASIC STRUCTURE OF A CLASS:

```
class ClassName {  
  
public:  
  
    // data members  
  
    // member functions  
  
};
```

Important Concepts:

- **Data Members:** Variables defined inside the class.
- **Member Functions:** Functions that operate on the data members.

- **Access Specifiers:**
 - o public → Accessible from outside the class.
 - o private → Accessible only inside the class.
- The . operator is used to access members of an object.

TASK 1 – CREATE AND USE A SIMPLE CLASS

Write the following program and execute it:

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int rollNo;

    void display() {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
    }
};

int main() {
    Student s1;
    s1.name = "Ali";
    s1.rollNo = 101;
    s1.display();
    return 0;
}
```

(1)

CONSTRUCTORS AND OBJECT CREATION

A constructor has the same name as the class, has no return type, and initializes object data automatically.

TASK 2 – ADD A CONSTRUCTOR

Modify the class as follows:

```
class Student {
public:
    string name;
    int rollNo;

    Student(string n, int r) {
```

```
        name = n;
        rollNo = r;
    }
    void display() {
        cout << name << " - " << rollNo << endl;
    }
};

int main() {
    Student s1("Sara", 102);
    s1.display();
    return 0;
}
```

(2)

POINTER FUNDAMENTALS

A pointer:

- Stores the address of a variable.
- Uses * for declaration.
- Uses & to get address.
- Uses * to access value (dereference).

TASK 3: BASIC POINTER EXAMPLE

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int* ptr = &x;

    cout << "Value of x: " << x << endl;
    cout << "Address of x: " << &x << endl;
    cout << "Pointer value: " << ptr << endl;
    cout << "Value via pointer: " << *ptr << endl;

    return 0;
}
```

Observation

- ptr stores address of x
- *ptr gives value stored at that address

DYNAMIC MEMORY (NEW AND DELETE)

In C++, memory can be allocated in two main areas:

1. Stack Memory

- Automatically managed by the compiler.
- Used for local variables and function calls.
- Memory is released automatically when the function ends.
- Fast but limited in size.

Example:

```
int x = 10;
```

Here, memory for x is created in the stack and automatically destroyed when the program leaves its scope.

2. Heap Memory (Dynamic Memory)

- Allocated manually during runtime.
- Memory remains allocated until it is explicitly released.
- Used when we need flexible or large memory allocation.
- Essential for dynamic data structures like Linked Lists, Trees, and Graphs.

To allocate memory in heap, we use the **new** operator:

```
int* p = new int;
```

This statement:

- Allocates memory in heap.
- Returns the address of that memory.
- Stores the address in pointer p.

To release the memory, we use the **delete operator**:

```
delete p;
```

Task 4: Dynamic Integer:

```
#include <iostream>
using namespace std;

int main() {
    int* p = new int;

    *p = 25;
```

```
cout << "Value: " << *p << endl;

delete p;

return 0;
}
```

(4)

POINTER TO OBJECT & ARROW OPERATOR

There are two ways to create objects:

1. STACK OBJECT

```
Student s1("Ali", 101);
```

```
//Access using:
```

```
s1.display();
```

2. HEAP OBJECT

```
Student* s2 = new Student("Ahmed", 105);
```

```
//Access using:
```

```
s2->display();
```

```
// -> is used to access members via pointer.
```

TASK 5: STACK VS HEAP OBJECT

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int rollNo;

    Student(string n, int r) {
        name = n;
        rollNo = r;
    }
}
```

```
void display() {  
    cout << name << " - " << rollNo << endl;  
}  
};  
  
int main() {  
  
    Student s1("Ali", 101);  
    s1.display();  
  
    Student* s2 = new Student("Ahmed", 105);  
    s2->display();  
  
    delete s2;  
  
    return 0;  
}
```

(5)

TASK 6: CREATE DYNAMIC STUDENT RECORDS

```
#include <iostream>  
using namespace std;  
  
class Student {  
public:  
    string name;  
    int rollNo;  
  
    Student() {  
        name = "";  
        rollNo = 0;  
    }  
  
    void setData(string n, int r) {  
        name = n;  
        rollNo = r;  
    }  
  
    void display() {  
        cout << name << " - " << rollNo << endl;  
    }  
};
```

```
    }  
};  
  
int main() {  
  
    int n;  
    cout << "Enter number of students: ";  
    cin >> n;  
  
    Student* students = new Student[n]; // dynamic array of objects  
  
    for(int i = 0; i < n; i++) {  
        string name;  
        int roll;  
  
        cout << "Enter name and roll: ";  
        cin >> name >> roll;  
  
        students[i].setData(name, roll);  
    }  
  
    cout << "\nStudent List:\n";  
    for(int i = 0; i < n; i++) {  
        students[i].display();  
    }  
  
    delete[] students; // very important  
  
    return 0;  
}
```

(6)

LAB EXERCISE:

1. Create a Car class with brand and model, constructor, and display function.
2. Create a dynamic integer array of size 5 using new. Take input, print values, and free memory.
3. Write a program that:
 - a. Dynamically creates two Car objects using new
 - b. Stores them using pointers
 - c. Displays their details using ->
 - d. Deletes both objects properly